

Final Technical Report

Sistem Manajemen Lab & Skrining Awal

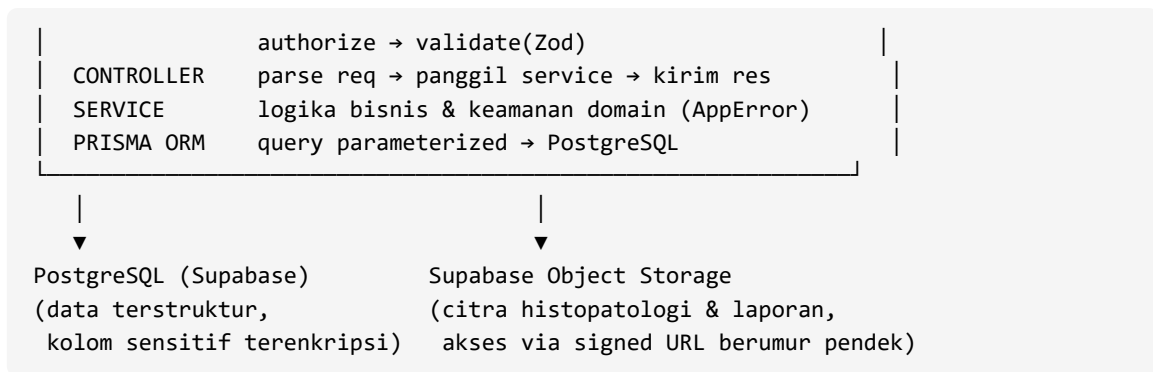
Tuberkulosis (TBC)

Projek Akhir Keamanan Informasi 2026
Kelompok 7

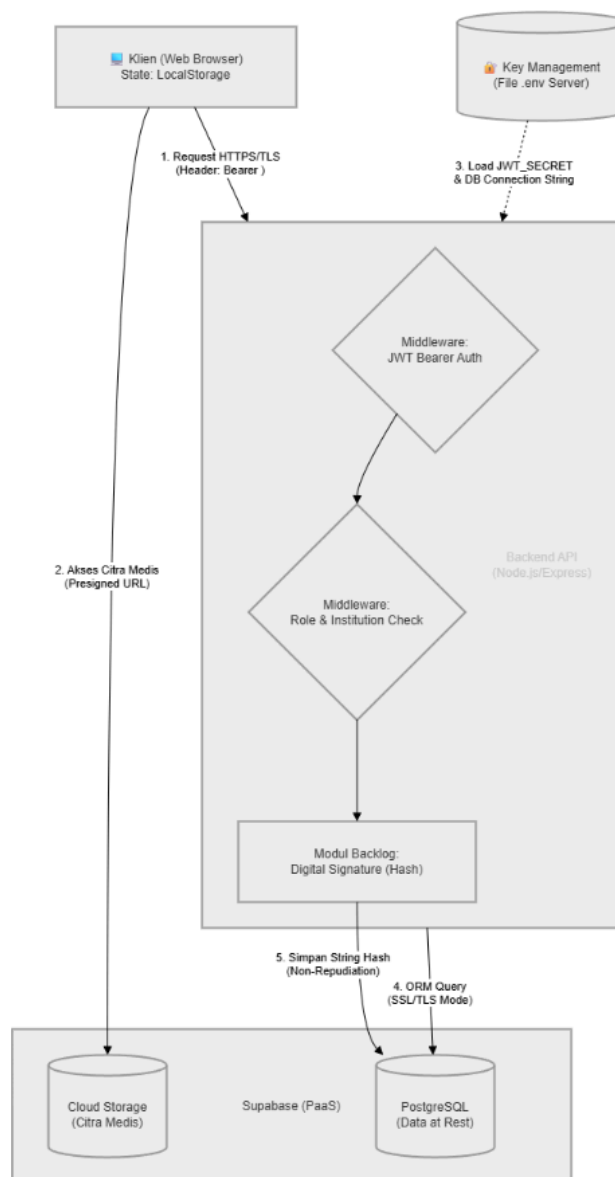
Walid Nadirul Ahnaf	G6401231109
Yasir	G6401231091
Cokorda Satria	G6401231064

Daftar Isi

1	Ringkasan Sistem	3
1.1	Latar Belakang	3
1.2	Tujuan & Ruang Lingkup	3
1.3	Status Implementasi	3
2	Arsitektur Sistem	3
2.1	Arsitektur Berlapis (Layered Architecture)	3
2.2	Trust Boundary	5
2.3	Tech Stack	6
3	Model Data & Keamanan Data	6
4	Kontrol Keamanan	7
4.1	Autentikasi: JWT (HS256)	7
4.2	Otorisasi: RBAC (Role-Based Access Control)	8
4.3	Hashing Password: bcrypt	9
4.4	Kontrol Akses Berbasis Institusi (Multi-tenant Isolation)	10
4.5	Validasi Input: Zod Schema	11
4.6	Manajemen Akun Terkontrol (Tanpa Registrasi Publik + First-Login)	11
4.7	Soft Delete & Content Masking pada Komentar	12
4.8	Presigned URL & Akses File Berbatas Waktu	13
4.9	Audit Trail: Append-Only Log	13
4.10	Tanda Tangan Digital pada Laporan Klinis	14
4.11	Reset Password Stateless & Aman	14
4.12	Enkripsi Field-Level: AES-256 (NIK & BPJS)	15
4.13	Rate Limiting: Mitigasi Brute-Force & Abuse	16
4.14	Ringkasan Pemetaan CIA Triad	17
5	API Reference	17
5.1	Matriks Endpoint × Peran	17
5.2	Format Response Standar	18
6	Pengujian & Evaluasi Keamanan	18
6.1	Unit Testing	18
6.2	Pengujian Skenario Serangan	19
6.3	Analisis Dampak Kinerja	20
7	Matriks Telusur (Traceability Matrix)	21
8	Batasan & Pengembangan Lanjutan	22
9	Lampiran	23
9.1	Struktur Direktori (ringkas)	23
9.2	Ringkasan Peran → Endpoint	23



Prinsip kunci dari desain ini adalah seluruh logika domain dan keamanan domain berada di service layer. Controller tidak boleh memuat logika bisnis; ia hanya menjembatani HTTP dengan service. Hal ini membuat kontrol keamanan (mis. `assertSameInstitution()`) terpusat dan tidak tersebar/duplikatif.



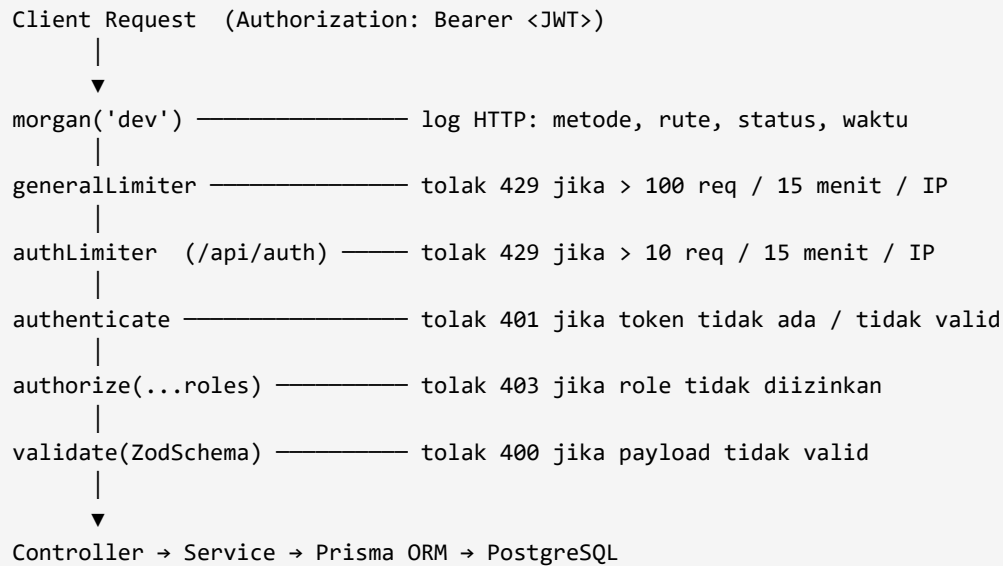
Gambar 1: Diagram arsitektur berlapis sistem.

Urutan pemasangan middleware global terlihat pada [src/index.ts](#):

```

app.use(morgan('dev'));           // logging HTTP (akuntabilitas jaringan)
app.use(express.json());
app.use(cors(corsOptions));      // batasi origin lewat CORS_ORIGIN
app.use(gerallimiter);           // rate limit global: 100 req / 15 menit / IP
app.use('/api/auth', authLimiter, authRoutes); // rate limit ketat: 10 req / 15
menit

```



2.2 Trust Boundary

Terdapat empat zona kepercayaan yang berbeda; setiap perpindahan antar-zona adalah titik penegakan keamanan:

Zona	Kepercayaan	Kontrol di perbatasan
Client (browser)	Tidak tepercaya	CORS, JWT wajib, validasi Zod
Backend API	Tepercaya (inti)	RBAC, isolasi institusi, audit log
Object Storage	Semi-tepercaya	Akses hanya via <i>presigned/signed URL</i> berumur pendek
PostgreSQL	Tepercaya (data at-rest)	Enkripsi field-level NIK/BPJS, query terparameterisasi (Prisma)

Tabel 1: Empat zona kepercayaan (trust boundary) dan kontrolnya.

2.3 Tech Stack

Layer	Teknologi	Versi
Runtime	Node.js (ES Modules)	LTS
Bahasa	TypeScript (strict mode)	^6.0
Framework	Express	^5.2
ORM	Prisma	^6.19
Database	PostgreSQL via Supabase	–
Autentikasi	JWT (jsonwebtoken) + bcrypt	–
Validasi	Zod	^4.3
Rate Limiting	express-rate-limit	^8.5
Kriptografi	Node crypto (AES-256, RSA-2048, SHA-256)	built-in
Logging	morgan	–
Storage	Supabase Object Storage	–
Dokumentasi API	Swagger (swagger-jsdoc + swagger-ui-express)	–
Testing	Vitest	^4.1

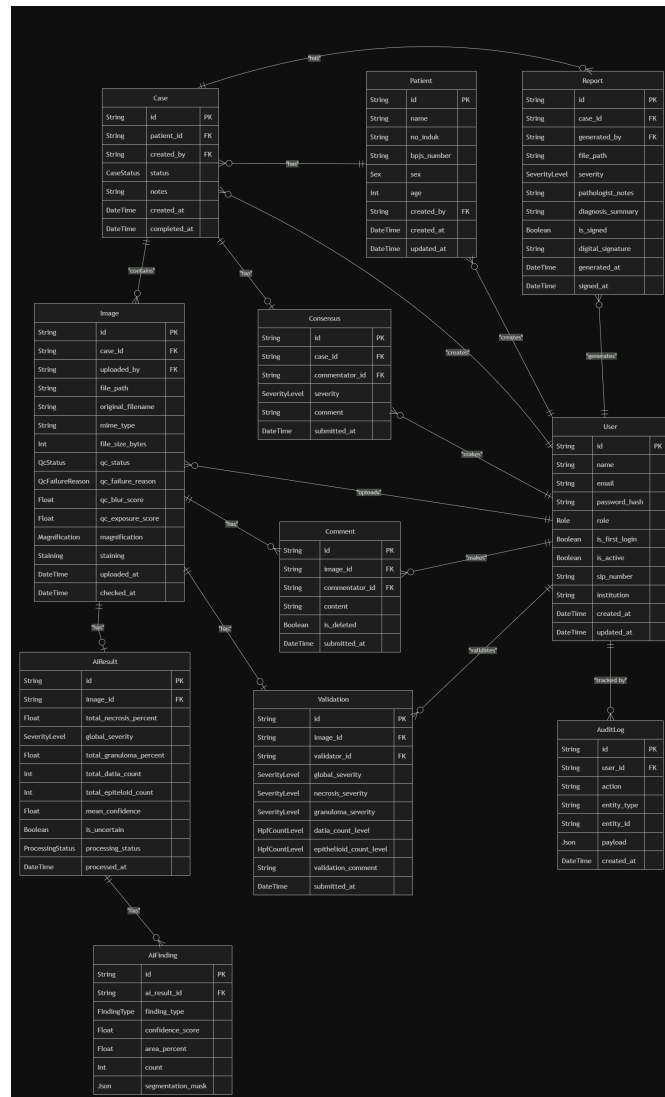
Tabel 2: Tumpukan teknologi (tech stack) backend.

3 Model Data & Keamanan Data

Sumber kebenaran skema: [prisma/schema.prisma](#). Tabel di bawah menyoroti properti keamanan tiap entitas, bukan seluruh field.

Entitas	Peran	Properti keamanan
User	Akun & peran	password_hash (bcrypt), role (RBAC), institution (isolasi), is_first_login, is_active (nonaktif, bukan hapus)
Patient	Identitas pasien	no_induk dan bpjs_number disimpan terenkripsi AES-256 (keduanya @unique)
Case	Kasus klinis	created_by → dasar pengecekan assertSameInstitution()
Image	Citra histopatologi	file_path di Object Storage; akses hanya via signed URL
Comment	Diskusi antar-dokter	is_deleted (soft delete); konten di-mask sebelum dikirim
Report	Laporan klinis	is_signed, digital_signature (SHA-256, tamper-evident), bersifat irreversible setelah ditandatangani
AuditLog	Jejak audit	append-only: user_id, action, entity_type, entity_id, payload, created_at

Tabel 3: Properti keamanan tiap entitas data.



Gambar 2: Entity Relationship Diagram sistem (sumber: 02_Design_Documents/ERD.png).

Catatan integritas data: entitas dengan jejak klinis tidak pernah dihapus permanen. User dinonaktifkan via `is_active = false`, Comment dihapus secara *soft*, dan AuditLog bersifat *append-only*. Ini menjaga keaslian rekam medis dan mendukung *non-repudiation*.

4 Kontrol Keamanan

Tiga belas kontrol berikut adalah inti laporan. Setiap kontrol disajikan dengan format seragam: (a) ancaman yang dimitigasi, (b) mekanisme, (c) implementasi (file dan cuplikan), (d) trade-off atau catatan, dan (e) pilar CIA. Seluruh cuplikan kode diambil langsung dari sumbernya.

4.1 Autentikasi: JWT (HS256)

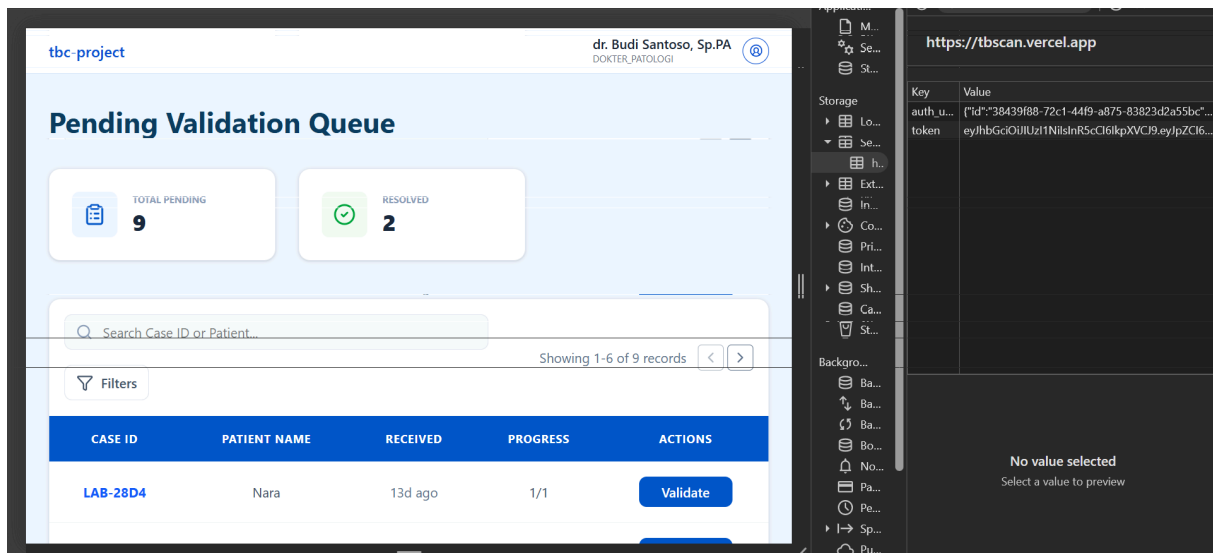
- **(a) Ancaman:** *Spoofing* identitas, pembajakan sesi. (OWASP API2:2023, Broken Authentication)
- **(b) Mekanisme:** autentikasi *stateless* berbasis *signed token*. Token login berlaku 1 hari; setiap request ke rute tertutup wajib membawa `Authorization: Bearer <token>`. JWT secret disimpan di environment variable, tidak *hardcoded*.

- (c) Implementasi: `src/utils/jwt.utils.ts`, `src/middlewares/authenticate.middleware.ts`:

```
export const generateToken = <TPayload extends object>({
  payload: TPayload,
  expiresIn: SignOptions["expiresIn"] = "1d"
}): string => jwt.sign(payload, getJwtSecret(), { expiresIn });
```

```
export const authenticate = (req, res, next): void => {
  const authHeader = req.headers.authorization;
  if (!authHeader?.startsWith("Bearer ")) {
    res.status(401).json({ status: "error", message: "Token autentikasi tidak ditemukan" });
    return;
  }
  const token = authHeader.split(" ")[1]!;
  try {
    req.user = verifyToken<LoginTokenPayload>(token);
    next();
  } catch {
    res.status(401).json({ status: "error", message: "Token tidak valid atau kedaluwarsa" });
  }
};
```

- (d) Trade-off: JWT *stateless* tidak punya mekanisme *revocation* sebelum kedaluwarsa (lihat §8). Algoritma HS256 dipilih (default jsonwebtoken) karena kunci simetris cukup untuk sistem single-issuer.
- (e) Pilar: Authentication, Confidentiality.



Gambar 3: Token JWT yang dibawa pada header Authorization: Bearer.

4.2 Otorisasi: RBAC (Role-Based Access Control)

- (a) Ancaman: *Elevation of Privilege*, yaitu pengguna peran rendah mengakses operasi peran lain. (OWASP API5:2023, Broken Function Level Authorization)

- **(b) Mekanisme:** *Principle of Least Privilege*. Tiga peran: OPERATOR_LAB, DOKTER_PATOLOGI, ADMIN_AI. Middleware authenticate memverifikasi JWT, lalu authorize(...roles) membatasi akses per peran secara eksplisit di tiap rute.
- **(c) Implementasi:** `src/middlewares/authenticate.middleware.ts`:

```
export const authorize =
  (...roles: Role[]) =>
  (req, res, next): void => {
    if (!req.user || !roles.includes(req.user.role as Role)) {
      res.status(403).json({ status: "error", message: "Anda tidak memiliki akses ke resource ini" });
      return;
    }
    next();
  };

```

- **(d) Trade-off:** otorisasi tingkat fungsi (per endpoint) dilengkapi otorisasi tingkat objek via isolasi institusi (§4.4) agar BOLA tertutup.
- **(e) Pilar:** Confidentiality, Integrity.

4.3 Hashing Password: bcrypt

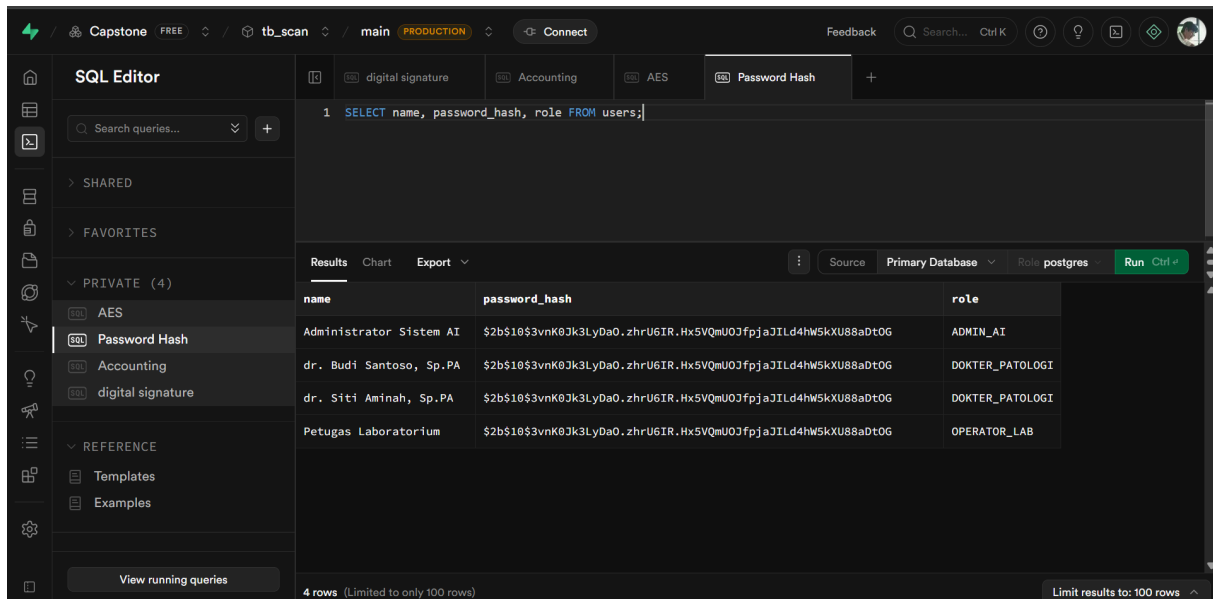
- **(a) Ancaman:** *Information Disclosure* kredensial jika DB bocor; brute-force. (OWASP API2:2023)
- **(b) Mekanisme:** password tidak pernah disimpan *plaintext*. *Adaptive hashing* bcrypt dengan salt rounds = 10 memperlambat brute-force secara sengaja.
- **(c) Implementasi:** `src/utils/hash.utils.ts`:

```
export const hashPassword = async (password: string): Promise<string> => {
  const salt = await bcrypt.genSalt(10);
  return bcrypt.hash(password, salt);
};

```

Saat ganti password, sistem memverifikasi password lama dan menolak password baru yang sama dengan yang aktif (lihat `auth.service.ts`).

- **(d) Trade-off:** *cost factor* 10 menambah latensi login (~72 ms, lihat §6.3). Ini adalah *trade-off* keamanan yang disengaja, bukan degradasi.
- **(e) Pilar:** Confidentiality, Authentication.



Gambar 4: Kolom `password_hash` di basis data dalam bentuk hash bcrypt, bukan plaintext.

4.4 Kontrol Akses Berbasis Institusi (Multi-tenant Isolation)

- **(a) Ancaman:** akses silang data antar fasilitas kesehatan; *Broken Object Level Authorization*. (OWASP API1:2023, BOLA)
- **(b) Mekanisme:** *data segregation*. Pengguna hanya dapat mengakses kasus/citra/laporan milik institusi yang sama. Pengecekan di service layer via `assertSameInstitution()`, yang menolak 403 bila institusi berbeda atau salah satu `institution` bernilai `null`.
- **(c) Implementasi:** `src/utils/access.utils.ts`:

```
export const assertSameInstitution = async (userAId: string, userBId: string) => {
  const [userA, userB] = await Promise.all([
    prisma.user.findUnique({ where: { id: userAId }, select: { institution: true } }),
    prisma.user.findUnique({ where: { id: userBId }, select: { institution: true } }),
  ]);
  if (!userA?.institution || !userB?.institution) {
    throw new AppError("Akses ditolak: institusi tidak terdaftar", 403);
  }
  if (userA.institution !== userB.institution) {
    throw new AppError("Akses ditolak: resource milik institusi lain", 403);
  }
};
```

Helper ini dipanggil di service yang mengakses data milik institusi, mis. `report.service.ts` dan `comment.service.ts`.

- **(d) Trade-off:** pengecekan berbasis logika bisnis (bukan *row-level security* DB), sehingga lebih fleksibel dan dapat diuji, namun harus konsisten dipanggil di setiap operasi mutasi.
- **(e) Pilar:** Confidentiality, Integrity.

4.5 Validasi Input: Zod Schema

- (a) **Ancaman:** *Tampering / injection* dan korupsi data dari payload anomali.
- (b) **Mekanisme:** setiap endpoint memiliki Zod schema eksplisit. Middleware `validate()` dijalankan sebelum controller, sehingga data kotor ditolak (400) sebelum menyentuh logika bisnis. TypeScript `strict: true` mencegah any membawa data tak aman.
- (c) **Implementasi:** `src/middlewares/validate.middleware.ts`:

```
export const validate =
  <TSchema extends ZodType>(schema: TSchema, source: "body" | "query" = "body") =>
  (req, res, next): void => {
    try {
      const parsed = schema.parse(req[source]);
      if (source === "body") req.body = parsed;
      else Object.defineProperty(req, "query", { value: parsed, writable: true,
        configurable: true });
      next();
    } catch (error: unknown) {
      if (error instanceof z.ZodError) {
        res.status(400).json({ status: "error", message:
          error.issues[0]?.message ?? "Payload tidak valid" });
        return;
      }
      res.status(500).json({ status: "error", message: "Terjadi kesalahan internal
        saat validasi request" });
    }
  };
};
```

Catatan: penggunaan Prisma ORM dengan query terparameterisasi sudah memitigasi SQL injection pada lapisan akses data; validasi Zod menambah pertahanan di gerbang masuk.

- (d) **Trade-off:** hanya pesan *issue* pertama yang dikembalikan (UX ringkas), bukan seluruh daftar error.
- (e) **Pilar:** Integrity.

4.6 Manajemen Akun Terkontrol (Tanpa Registrasi Publik + First-Login)

- (a) **Ancaman:** penyalahgunaan pendaftaran publik; *credential stuffing* pada kredensial bawaan. (OWASP API2:2023)
- (b) **Mekanisme:** pendekatan *zero-trust* terhadap pembuatan akun. Tidak ada endpoint `POST /register`; akun dibuat admin langsung ke DB. Login pertama wajib ganti password (`is_first_login = true`). Akun berjejak klinis dinonaktifkan, tidak dihapus.
- (c) **Implementasi:** `src/services/auth.service.ts`:

```
export const updateCredential = async (email, currentPassword, newPassword,
  confirmPassword) => {
  const user = await prisma.user.findUnique({ where: { email } });
  if (!user) throw new AppError("Email atau password tidak valid", 401);
  if (!user.is_first_login) throw new AppError("Akun ini tidak memerlukan update
    credential awal", 400);
  // ... verifikasi password lama, cek konfirmasi, larang password sama, lalu hash
  & simpan
};
```

- **(d) Trade-off:** penyediaan akun bersifat manual/administratif, menukar kemudahan onboarding demi memperkecil *attack surface*.
- **(e) Pilar:** Authentication, Confidentiality.

Gambar 5: Pengguna diwajibkan mengganti password bawaan saat login pertama.

4.7 Soft Delete & Content Masking pada Komentar

- **(a) Ancaman:** hilangnya jejak diskusi klinis; *Repudiation* / *Tampering* rekam medis.
- **(b) Mekanisme:** komentar tidak dihapus fisik, melainkan ditandai `is_deleted = true`, dan kontennya di-mask menjadi teks netral sebelum dikirim ke client. Hanya penulis yang dapat menghapus komentarnya sendiri.
- **(c) Implementasi:** `src/services/comment.service.ts`:

```
export const DELETED_CONTENT = "pesan ini telah dihapus";

export const maskDeletedContent = (comment: { content: string; is_deleted: boolean }) =>
  comment.is_deleted ? { ...comment, content: DELETED_CONTENT } : comment;

export const deleteComment = async (commentId: string, requesterId: string) => {
  const comment = await prisma.comment.findUnique({ where: { id: commentId } });
  if (!comment) throw new AppError("Komentar tidak ditemukan", 404);
  if (comment.is_deleted) throw new AppError("Komentar sudah dihapus", 409);
  if (comment.commentator_id !== requesterId)
    throw new AppError("Akses ditolak: hanya penulis komentar yang dapat menghapusnya", 403);
  return prisma.comment.update({ where: { id: commentId }, data: { is_deleted: true } });
};
```

- **(d) Trade-off:** data terhapus tetap menempati basis data; diterima demi integritas riwayat.
- **(e) Pilar:** Integrity, Non-Repudiation.

4.8 Presigned URL & Akses File Berbatas Waktu

- **(a) Ancaman:** *Information Disclosure* citra/laporan medis melalui pembagian tautan publik. (OWASP API1:2023)
- **(b) Mekanisme:** berkas medis tidak pernah diekspos publik. Akses memakai *time-limited signed URL*: upload berlaku 1 jam, sedangkan view/download berlaku 15 menit (900 detik) dan harus diminta ulang setelah kedaluwarsa.
- **(c) Implementasi:** `src/services/storage/supabase.storage.ts`:

```
async createPresignedUploadUrl(filePath: string, expiresInSeconds = 3600):  
  Promise<string> { /* ... */ }  
  
async createSignedViewUrl(filePath: string, expiresInSeconds = 900):  
  Promise<string> {  
    const { data, error } = await  
      supabase.storage.from(BUCKET).createSignedUrl(filePath, expiresInSeconds);  
    if (error || !data?.signedUrl) throw new AppError("Gagal membuat URL view", 500);  
    return data.signedUrl;  
  }  
}
```

- **(d) Trade-off:** durasi pendek meningkatkan keamanan namun menuntut client me-*refresh* URL.
- **(e) Pilar:** Confidentiality.

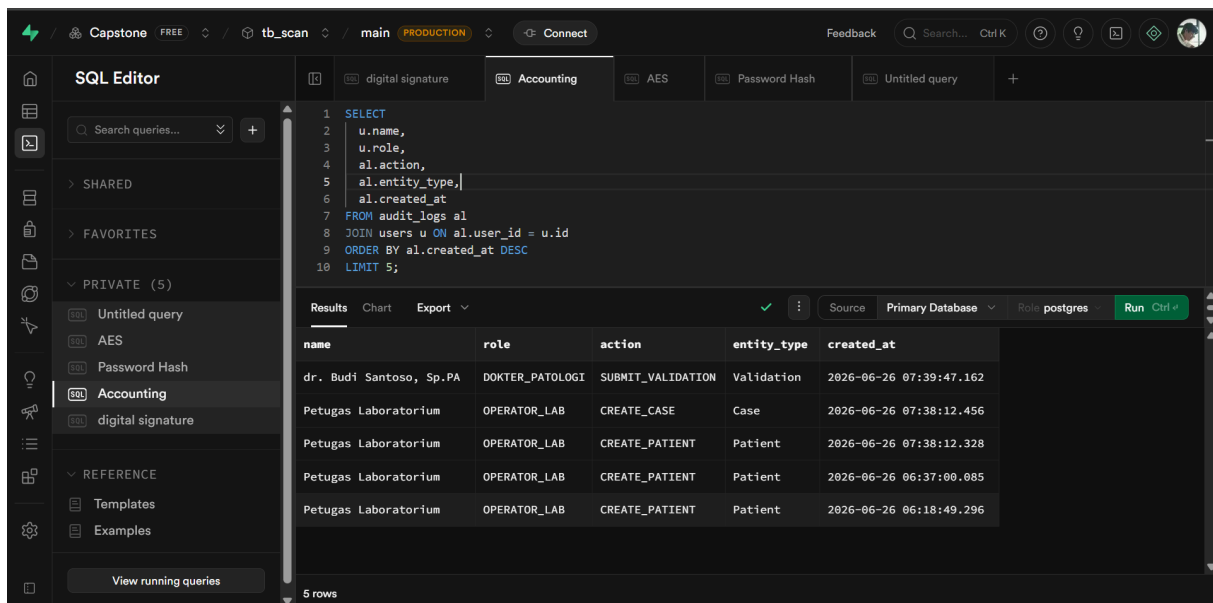
4.9 Audit Trail: Append-Only Log

- **(a) Ancaman:** *Repudiation*, yaitu pelaku menyangkal tindakannya.
- **(b) Mekanisme:** setiap aksi mutasi penting dicatat ke AuditLog. Log bersifat *append-only* (tidak ada UPDATE/DELETE). Penulisan bersifat non-blocking: kegagalan log tidak menggagalkan transaksi bisnis.
- **(c) Implementasi:** `src/utils/audit.utils.ts`:

```
export const writeAuditLog = async (userId, action, entityType, entityId, payload?)  
=> {  
  try {  
    await prisma.auditLog.create({  
      data: { user_id: userId, action, entity_type: entityType, entity_id:  
        entityId,  
        ...(payload !== undefined && { payload: payload as  
          Prisma.InputJsonValue }) },  
    });  
  } catch (err) {  
    console.error("[AuditLog] Gagal menulis audit log:", err); // non-blocking  
  }  
};
```

Dipanggil pada aksi mutasi seperti CREATE_PATIENT, UPDATE_PATIENT, GENERATE_REPORT, SIGN_REPORT. Selain audit tingkat aplikasi, middleware morgan mencatat seluruh request HTTP (rute, metode, status, waktu) untuk deteksi anomali & forensik.

- **(d) Trade-off:** desain non-blocking memilih *availability* di atas jaminan tulis log; risiko kehilangan sebagian entri saat DB bermasalah, dengan kompensasi log error di konsol.
- **(e) Pilar:** Non-Repudiation, Integrity.



Gambar 6: Contoh baris AuditLog (append-only) di basis data.

4.10 Tanda Tangan Digital pada Laporan Klinis

- **(a) Ancaman:** *Tampering* isi laporan; *Repudiation* atas laporan yang diterbitkan.
- **(b) Mekanisme:** saat finalisasi, *snapshot* laporan di-hash SHA-256, lalu hash tersebut diikat dengan ID laporan, identitas penandatanganan, dan waktu menjadi segel *tamper-evident*. Setelah ditandatangani, laporan tidak dapat diubah atau ditandatangani ulang (*irreversible*).
- **(c) Implementasi:** `src/services/report.service.ts`:

```

const snapshotBytes = await storage.downloadFile(report.file_path);
const contentHash =
  crypto.createHash("sha256").update(snapshotBytes).digest("hex");

// Ikat hash konten + ID laporan + identitas penandatanganan + waktu
const signatureInput = [contentHash, report.id, patologiId,
  signedAt.toISOString()].join("|");
const digitalSignature =
  crypto.createHash("sha256").update(signatureInput).digest("hex");

```

- **(d) Trade-off:** skema saat ini berbasis hash SHA-256 (simetris), sehingga mampu mendeteksi perubahan, tetapi belum mendukung verifikasi independen pihak ketiga. Utilitas asimetris RSA-2048 (RSA-SHA256) sudah disiapkan di `src/utls/signature.utls.ts` sebagai rencana penyempurnaan (lihat §8).
- **(e) Pilar:** Integrity, Non-Repudiation.

4.11 Reset Password Stateless & Aman

- **(a) Ancaman:** pencurian token reset dari DB; penyalahgunaan token login sebagai token reset. (OWASP API2:2023)
- **(b) Mekanisme:** token reset adalah JWT tersendiri (purpose: "reset_password") berlaku 15 menit, diverifikasi secara kriptografis tanpa *lookup* DB. Token login biasa ditolak bila dipakai untuk reset (validasi field purpose).

- (c) **Implementasi:** `src/services/auth.service.ts`, `src/types/auth.types.ts`:

```
export const resetPassword = async (token: string, newPassword: string) => {
  const decoded = verifyToken<PasswordResetTokenPayload>(token);
  if (decoded.purpose !== RESET_PASSWORD_PURPOSE)
    throw new AppError("Token reset password tidak valid", 401);
  // ... cek user, larang password sama, hash & simpan
};
```

- (d) **Trade-off:** karena *stateless*, token tidak bisa dicabut sebelum 15 menit berakhir; jendela waktu sengaja dibuat pendek untuk membatasi risiko.
- (e) **Pilar:** Authentication, Confidentiality.

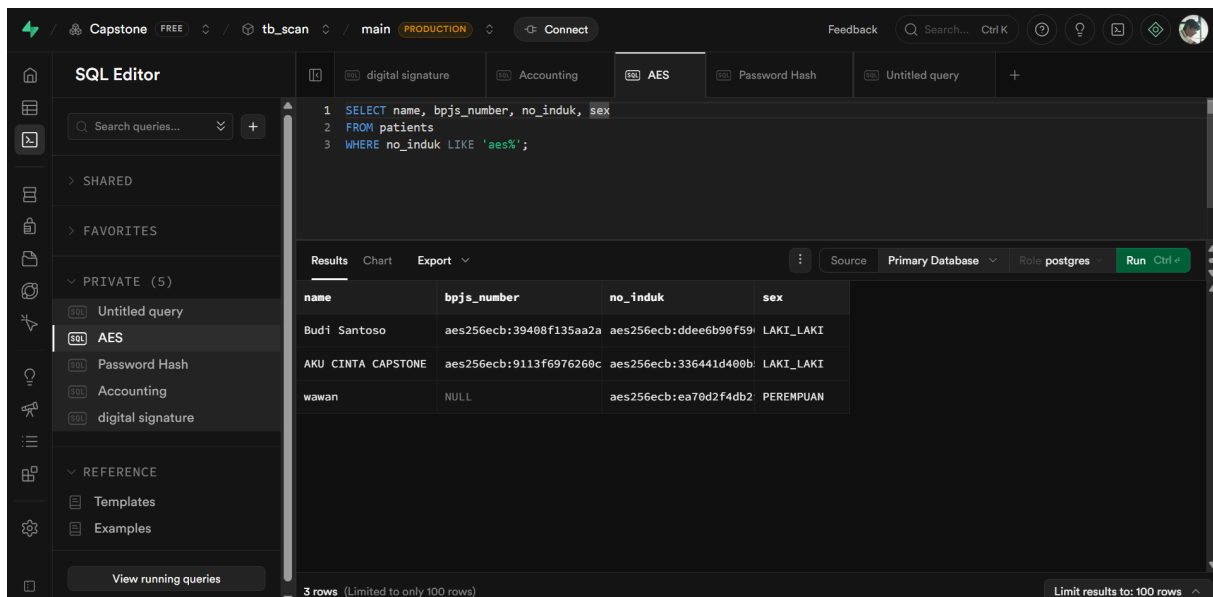
4.12 Enkripsi Field-Level: AES-256 (NIK & BPJS)

- (a) **Ancaman:** *Information Disclosure* identitas pasien bila DB bocor (*encryption at rest* tingkat kolom).
- (b) **Mekanisme:** `no_induk` (NIK) dan `bpjs_number` dienkripsi AES-256-ECB sebelum disimpan dan didekripsi otomatis saat dibaca. Mode ECB (deterministik) dipilih agar plaintext sama menghasilkan ciphertext sama, sehingga constraint `@unique` PostgreSQL tetap berfungsi. Idempoten: nilai ber-prefix `aes256ecb`: tidak dienkripsi ganda.
- (c) **Implementasi:** `src/utils/crypto.utils.ts`, dipakai di `src/services/patient.service.ts`:

```
export const encryptField = (plaintext: string): string => {
  if (plaintext.startsWith(PREFIX)) return plaintext; // idempoten
  const key = getAesKey(); // 32 byte dari
  AES_SECRET_KEY (64 hex)
  const cipher = crypto.createCipheriv("aes-256-ecb", key, null);
  const encrypted = Buffer.concat([cipher.update(plaintext, "utf-8"),
  cipher.final()]);
  return `${PREFIX}${encrypted.toString("hex")}`;
};
```

Trade-off eksplisit: ECB tidak menyembunyikan pola pada data identik. Diterima karena field identifikasi pendek (NIK 16 digit, BPJS 13 digit) dan demi mempertahankan *unique constraint*. Kunci berasal dari `AES_SECRET_KEY`, tidak *hardcoded*.

- (e) **Pilar:** Confidentiality.



Gambar 7: Kolom `no_induk` dan `bpjs_number` tersimpan sebagai ciphertext AES-256.

4.13 Rate Limiting: Mitigasi Brute-Force & Abuse

- (a) **Ancaman:** *Denial of Service*, brute-force, abuse API. (OWASP API4:2023, Unrestricted Resource Consumption)
- (b) **Mekanisme:** dua pembatas laju, yaitu `generalLimiter` (100 req / 15 menit per IP untuk seluruh API) dan `authLimiter` (10 req / 15 menit khusus `/api/auth`, lebih ketat untuk proteksi kredensial). Pelanggaran dibalas HTTP 429 dengan format konsisten.
- (c) **Implementasi:** `src/middlewares/rate-limit.middleware.ts`:

```
export const generalLimiter = ratelimit({
  windowMs: 15 * 60 * 1000, max: 100, standardHeaders: true, legacyHeaders: false,
  message: "Terlalu banyak permintaan, coba lagi dalam 15 menit", handler:
  ratelimitResponse,
});

export const authLimiter = rateLimit({
  windowMs: 15 * 60 * 1000, max: 10, standardHeaders: true, legacyHeaders: false,
  message: "Terlalu banyak percobaan login, coba lagi dalam 15 menit", handler:
  ratelimitResponse,
});
```

- (d) **Trade-off:** pembatasan per-IP dapat memberi efek ke pengguna di belakang NAT bersama; ambang dipilih cukup longgar untuk pemakaian normal.
- (e) **Pilar:** Availability.

4.14 Ringkasan Pemetaan CIA Triad

Pilar	Kontrol yang berkontribusi
Confidentiality	JWT (§4.1), RBAC (§4.2), bcrypt (§4.3), Isolasi institusi (§4.4), No public register (§4.6), Signed URL (§4.8), Reset stateless (§4.11), AES-256 NIK/BPJS (§4.12)
Integrity	Validasi Zod (§4.5), Soft delete (§4.7), Audit log append-only (§4.9), Tanda tangan digital (§4.10)
Availability	Rate limiting (§4.13), audit log non-blocking (§4.9)
Non-Repudiation	Audit trail (§4.9), Tanda tangan digital (§4.10), Soft delete (§4.7)
Authentication	JWT + bcrypt (§4.1, §4.3), First-login (§4.6), Reset token stateless (§4.11)

Tabel 4: Pemetaan kontrol keamanan ke pilar CIA Triad.

5 API Reference

5.1 Matriks Endpoint × Peran

Auth (rute publik, dilindungi authLimiter):

Method	Endpoint	Deskripsi
POST	/api/auth/login	Login → JWT token
POST	/api/auth/update-credentials	Ganti password (first login)
POST	/api/auth/forgot-password	Generate reset token
POST	/api/auth/reset-password	Reset password via token

Tabel 5: Endpoint autentikasi.

Operator Lab (OPERATOR_LAB):

Method	Endpoint
GET/POST	/api/patients
GET/PATCH	/api/patients/:id
GET/POST	/api/cases
POST	/api/cases/:id/images/presigned-urls
POST	/api/cases/:id/images/confirm
POST	/api/cases/:id/submit
DELETE	/api/images/:id

Tabel 6: Endpoint Operator Lab.

Dokter Patologi (DOKTER_PATOLOGI; sebagian berbagi dengan OPERATOR_LAB):

Method	Endpoint	Akses
GET	/api/review/queue	DOKTER_PATOLOGI
GET	/api/review/resolved	DOKTER_PATOLOGI
GET	/api/review/cases/:caseId/images	DOKTER_PATOLOGI
GET	/api/review/cases/:caseId/images/:id	DOKTER_PATOLOGI, OPERATOR_LAB
POST	/api/images/:id/validate	DOKTER_PATOLOGI
POST	/api/images/:id/comments	DOKTER_PATOLOGI, OPERATOR_LAB
POST	/api/cases/:id/consensus	DOKTER_PATOLOGI

Tabel 7: Endpoint Dokter Patologi.

Laporan Klinis (DOKTER_PATOLOGI):

Method	Endpoint	Deskripsi
POST	/api/reports	Generate laporan dari kasus RESOLVED
GET	/api/reports/:id	Ambil laporan + signed URL download (15 menit)
PATCH	/api/reports/:id/finalize	Tanda tangan digital (irreversible)

Tabel 8: Endpoint Laporan Klinis.

Dokumentasi: endpoint GET /api-docs menyajikan Swagger UI interaktif.

5.2 Format Response Standar

```
// Sukses
{ "status": "success", "message": "...", "data": { } }
// Sukses + pagination
{ "status": "success", "message": "...", "data": [ ], "meta": { "total": 0, "page": 1, "limit": 10 } }
// Error (via AppError → error-handler middleware)
{ "status": "error", "message": "Pesan error yang jelas" }
```

6 Pengujian & Evaluasi Keamanan

6.1 Unit Testing

Pengujian unit menggunakan Vitest, mencakup tiga area di `src/__tests__`:

- **Services:** auth, case, comment, consensus, image, patient, review, validation, dan storage/supabase.
- **Utils:** crypto (enkripsi AES), hash (bcrypt), jwt, signature (RSA), access (isolasi institusi), pagination.
- **Validations:** schema Zod tiap domain.

Rincian hasil eksekusi terdapat pada [05_Testing/Hasil Unit Testing.pdf](#). Cakupan ini memastikan kontrol keamanan kritis (kripto, hashing, JWT, isolasi) terverifikasi secara otomatis, bukan hanya secara manual.

6.2 Pengujian Skenario Serangan

Delapan skenario merepresentasikan vektor ancaman utama. Tiap skenario mendefinisikan kondisi payload anomali, kode HTTP yang diharapkan, dan kriteria lulus yang objektif.

ID	Skenario	Payload / Kondisi Uji	Eksp.	Aktual	Hasil
TC-01	Akses rute tertutup tanpa token	Header Authorization: null	401	401	Lulus
TC-02	Token JWT kedaluwarsa	Token berumur > 24 jam	401	401	Lulus
TC-03	SQL Injection pada form auth	email: "admin' OR 1=1 --"	401	401	Lulus
TC-04	Brute-force / spam request login	> 100 request beruntun < 15 menit	429	429	Lulus
TC-05	Pelanggaran wewenang RBAC	GET /api/review/queue dgn token Operator Lab	403	403	Lulus
TC-06	Pelanggaran isolasi institusi	Akses pasien institusi A pakai token institusi B	403	403	Lulus
TC-07	Bypass validasi Zod	POST pasien tanpa atribut no_induk	400	400	Lulus
TC-08	Signed URL kedaluwarsa	Akses tautan citra setelah > 15 menit	403	403	Lulus

Tabel 9: Hasil pengujian delapan skenario serangan.

Seluruh request tidak sah berhasil diintersepsi middleware sebelum mencapai *core business logic*. Dua mekanisme disorot khusus: rate limiting (TC-04 menghasilkan 429 saat simulasi beban tinggi) dan RBAC (TC-05 menghasilkan 403 saat simulasi *privilege escalation*).

```

Test Cases

✓ LULUS TC-01 Akses rute tertutup tanpa token [HTTP 401]
✓ LULUS TC-02 Token JWT kedaluwarsa / signature tidak valid [HTTP 401]
✓ LULUS TC-03 SQL Injection pada form auth (bukan 200) [HTTP 400 ≠ 200]
✓ LULUS TC-05 Pelanggaran RBAC (Operator akses endpoint DOKTER_PATOLOGI) [HTTP 403]
i TC-06 Setup tambahan diperlukan:
1. Buat operator dari institusi berbeda (lihat komentar di atas)
2. Login dan dapatkan tokennya
3. Jalankan: OPERATOR_B_TOKEN=<token> ./test-security.sh
⚠ SKIP TC-06 Pelanggaran isolasi institusi [OPERATOR_B_TOKEN tidak di-set]
✓ LULUS TC-07 Bypass validasi Zod (POST pasien tanpa no_induk) [HTTP 400]
i TC-08 Verifikasi manual diperlukan:
1. Upload citra → dapatkan view_url dari GET /api/cases/:id/images
2. Tunggu > 15 menit
3. Akses URL tersebut → harapkan HTTP 400/403 dari Supabase Storage
⚠ SKIP TC-08 Signed URL kedaluwarsa [Tidak dapat diotomasi (perlu tunggu > 15 mnt)]

⚠ TC-04 Brute-force test akan menghabiskan auth rate-limit selama 15 mnt.
Lanjutkan? (y/N) y
Mengirim request login berulang ke /api/auth/login...
Request #1 → HTTP 401
Request #2 → HTTP 401
Request #3 → HTTP 401
Request #4 → HTTP 401
Request #5 → HTTP 401
Request #6 → HTTP 401
Request #7 → HTTP 429
✓ LULUS TC-04 Brute-force login diblokir rate-limiter [HTTP 429]

```

Gambar 8: Respons HTTP 401/403/429 yang dikembalikan server pada skenario uji.

6.3 Analisis Dampak Kinerja

Pengukuran membandingkan kondisi sebelum dan sesudah penambahan lapisan keamanan di lingkungan pengembangan lokal.

Metrik	Sebelum	Sesudah	Δ	Δ (%)
Login RT	219.4 ms	291.8 ms	+72.4 ms	+33.0%
Get Data RT	0.8 ms	1.4 ms	+0.6 ms	+75.0%
Upload RT	N/A	N/A	N/A	N/A
CPU Load/Login	N/A	72 ms/op	+72 ms	N/A
Memory Heap	6.6 MB	8.1 MB	+1.5 MB	+22.7%

Tabel 10: Dampak kinerja sebelum vs sesudah penambahan lapisan keamanan.

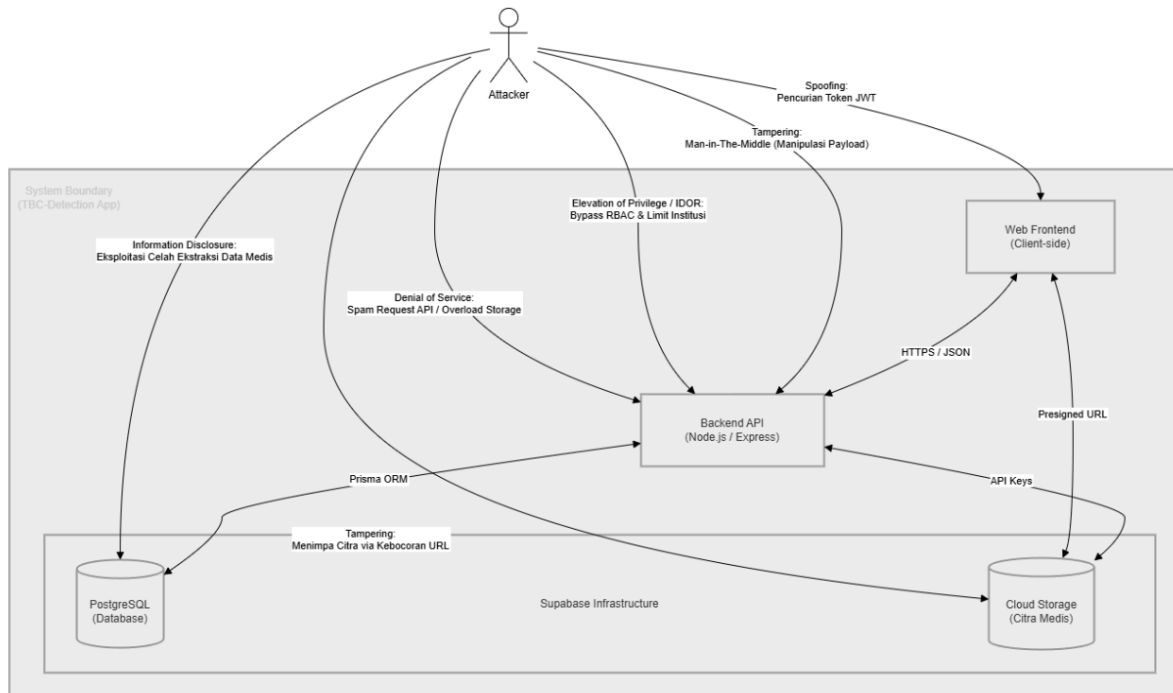
Kenaikan waktu respons login sebesar +33,0% adalah konsekuensi terencana dari bcrypt (*cost factor* 10), yang memang dirancang menuntut komputasi CPU intensif untuk memperlambat brute-force. Tambahan memori heap sebesar +1,5 MB dialokasikan untuk middleware keamanan yang berjalan simultan (rate limiter, verifikasi JWT, audit log). Secara keseluruhan *overhead* tergolong minimal dan dalam batas toleransi, sehingga keamanan tinggi tercapai tanpa mengorbankan ketersediaan secara signifikan.

7 Matriks Telusur (Traceability Matrix)

Matriks ini mengikat ancaman, kontrol, file, dan bukti uji, sehingga keamanan dapat dibuktikan, bukan sekadar diklaim.

Ancaman (STRIDE / OWASP)	Aset terdampak	Kontrol (§4)	File implementasi	Bukti uji
Spoofing identitas (API2)	Sesi pengguna	JWT auth (§4.1)	jwt.utils.ts, authenticate.middleware.ts	jwt.utils.test.ts; TC-01, TC-02
Elevation of Privilege (API5)	Operasi klinis	RBAC (§4.2)	authenticate.middleware.ts	TC-05
Information Disclosure kredensial (API2)	Password	bcrypt (§4.3)	hash.utils.ts	hash.utils.test.ts
BOLA lintas institusi (API1)	Data kasus/pasien	Isolasi institusi (§4.4)	access.utils.ts	access.utils.test.ts; TC-06
Tampering / Injection	Semua input	Validasi Zod (§4.5)	validate.middleware.ts, validations/	validation.test.ts; TC-07
Spoofing via register/credential stuffing (API2)	Akun	No register + first-login (§4.6)	auth.service.ts	auth.service.test.ts
Repudiation / Tampering diskusi	Komentar klinis	Soft delete + mask (§4.7)	comment.service.ts	comment.service.test.ts
Information Disclosure berkas (API1)	Citra & laporan	Signed URL (§4.8)	supabase.storage.ts	supabase.storage.test.ts; TC-08
Repudiation aksi	Jejak aktivitas	Audit log append-only (§4.9)	audit.utils.ts	(manual + log DB)
Tampering laporan	Laporan klinis	Tanda tangan digital (§4.10)	report.service.ts, signature.utils.ts	signature.utils.test.ts
Spoofing token reset (API2)	Alur reset password	Reset stateless (§4.11)	auth.service.ts, auth.types.ts	auth.service.test.ts
Information Disclosure identitas	NIK / BPJS	AES-256 field-level (§4.12)	crypto.utils.ts, patient.service.ts	crypto.utils.test.ts
Denial of Service (API4)	Ketersediaan API	Rate limiting (§4.13)	rate-limit.middleware.ts	TC-04

Tabel 11: Matriks telusur ancaman, kontrol, file, dan bukti uji.



Gambar 9: Ringkasan threat modelling sistem.

8 Batasan & Pengembangan Lanjutan

Sebagai bentuk objektivitas, berikut batasan sistem saat ini beserta ancaman residualnya:

1. Tanda tangan digital masih simetris (SHA-256). Skema ini mampu mendeteksi perubahan isi laporan, tetapi belum mendukung verifikasi independen pihak ketiga. Rencana: migrasi ke RSA-2048 (RSA-SHA256) yang utilitasnya sudah tersedia di `signature.utils.ts` dan field `digital_signature` sudah disiapkan di skema.
2. AES-256-ECB tidak menyembunyikan pola data identik. Dipilih demi mempertahankan `@unique` constraint pada NIK/BPJS. Alternatif masa depan: AES-GCM dengan strategi indeks *blind index* terpisah untuk pencarian.
3. JWT stateless tanpa refresh/revocation. Token tidak dapat dicabut sebelum kedaluwarsa (login 1 hari, reset 15 menit). Rencana: mekanisme *refresh token* dan daftar cabut (*denylist*).
4. bcrypt cost factor 10. Cukup untuk saat ini; perlu ditinjau berkala seiring naiknya daya komputasi penyerang.
5. QC citra masih di-mock (otomatis PASSED) dan inferensi AI via seeding, sedangkan endpoint dan *state machine* sudah siap untuk integrasi nyata.
6. Pengujian kinerja masih lingkungan lokal. Belum diuji pada infrastruktur produksi skala besar dengan ribuan request konkuren.

9 Lampiran

9.1 Struktur Direktori (ringkas)

```
Projek Akhir KI/
├── 01_Proposal_&_Analisis/      # Proposal teknis & threat modelling
├── 02_Design_Documents/        # ERD, Architecture Diagram, Testing Plan
├── 03_Source_Code/backend/
│   ├── prisma/                 # schema.prisma, migrasi, seed
│   ├── scripts/                # generate-rsa-keys.ts
│   └── src/
│       ├── config/             # prisma, supabase, swagger
│       ├── routes/             # endpoint per domain
│       ├── controller/         # handler HTTP
│       ├── services/           # logika bisnis & keamanan domain
│       │   └── storage/         # abstraksi Supabase Storage
│       ├── validations/        # Zod schema per domain
│       └── middlewares/        # authenticate, validate, rate-limit, error-
handler
│   └── utils/                  # jwt, hash, crypto (AES), signature (RSA), audit,
access
│   └── __tests__/              # unit test (Vitest)
├── 04_Reports_&_Paper/         # Laporan akhir, paper ilmiah, progress report
└── 05_Testing/                # Hasil unit testing
```

9.2 Ringkasan Peran → Endpoint

Peran (enum kode)	Label	Akses utama
OPERATOR_LAB	Operator/Teknisi Lab	Kelola pasien & kasus, upload citra, submit ke antrian
DOKTER_PATOLOGI	Dokter Patologi	Review & validasi AI, komentar, consensus, generate & tanda tangan laporan
ADMIN_AI	Admin AI	Monitoring sistem AI (belum diimplementasi)

Tabel 12: Ringkasan peran dan akses utamanya.

Catatan: pada paper, OPERATOR_LAB dirujuk sebagai “Teknisi Lab”. Di dokumen ini digunakan nama enum kode agar konsisten dengan implementasi (Role enum di `schema.prisma`).